

BigPower: Hierarchical Source-Level Module Power Estimation for CPUs with Large Language Models

Honghua Zhu

State Key Lab of Processors, Institute
of Computing Technology, Chinese
Academy of Sciences
Beijing, China
University of Chinese Academy of
Sciences
Beijing, China
zhuhonghua22s@ict.ac.cn

Chunjie Luo*

State Key Lab of Processors, Institute
of Computing Technology, Chinese
Academy of Sciences
Beijing, China
luochunjie@ict.ac.cn

Jianfeng Zhan

State Key Lab of Processors, Institute
of Computing Technology, Chinese
Academy of Sciences
Beijing, China
zhanjianfeng@ict.ac.cn

Abstract

Accurate power estimation is important for understanding and optimizing CPU power behavior, yet practical workflows often rely on simulation-derived information or post-silicon analysis. In this work, we present BigPower, a hierarchical source-level surrogate model for fine-grained module-level power estimation during CPU design. BigPower leverages large language model-based representations together with architectural hierarchy, module connectivity, configuration parameters, and workload context to estimate module-level power consumption directly from source-level design information, without requiring additional simulation during inference. Experimental results in the open-source XiangShan processor family demonstrate practical fine-grained power estimation across diverse configurations and workloads, offering an efficient alternative to conventional simulation-based workflows.

1 Introduction

Improving CPU power efficiency has become increasingly important as modern computing workloads continue to grow in scale and complexity.

There are three objectives for CPU power consumption evaluation: 1) Accuracy. Precise evaluation is critical, as inaccuracies can significantly misguide CPU design decisions. 2) Cost. Minimizing evaluation cost, particularly time, is essential to accelerate the iteration cycle of CPU design optimization. 3) Granularity. This encompasses both spatial and temporal dimensions. Spatial granularity refers to whether the estimation is performed on the entire chip or on specific modules within it. In terms of temporal granularity, per-cycle power estimation is a research hot spot.

Gate-level power analysis is generally considered the most accurate approach for power sign-off, which utilizes power-characterized standard cell libraries with commercial tools such as Synopsys PrimeTime PX (PTPX) by feeding in gate-level netlists and simulation results. RTL-level power modeling tools, such as PowerArtist, PowerPro, Apollo [43], Primal [54], and PACE [2], utilize RTL simulation traces as training features for power consumption estimation, but their accuracy is relatively lower. Positioned between gate-level and RTL-level modeling, GRANNITE [51] utilizes two inputs: a graph representation of the gate-level netlist and dynamic toggle

rates of input ports and register outputs obtained via RTL simulation.

Architecture-level models, such as CACTI [20], Wattch [3], McPAT [18], McPAT-PVT [35], McPAT-Monolithic [12], McPAT-Calib [47], and FirePower [49], typically trade estimation accuracy for lower evaluation cost and higher scalability. In terms of evaluation cost, however, the opposite is true: the time cost increases sequentially for architecture-level, RTL-level, and gate-level evaluations. Additionally, RTL-level and gate-level evaluations allow for fine-grained power modeling, while architecture-level modeling tends to be relatively coarse-grained.

Consequently, many practical power estimation workflows still rely on simulation-derived activity information, such as architectural event statistics or RTL/gate-level switching activities, to construct and calibrate power models. The finer the simulation granularity, the more reliable and detailed the captured information becomes, leading to higher accuracy and finer-grained modeling. However, this comes at the expense of increased time costs, which may slow iterative CPU optimization and evaluation.

Besides, CPU design space exploration requires power models to be capable of making predictions for unseen designs. In contrast, frameworks such as Primal [54] and Apollo [43] are only modeled for fixed designs; when the design changes, the models must be reconstructed and retrained. While McPAT-Calib [47] utilizes CPU configuration parameters and dynamic features generated by the Gem5 simulator to provide predictions for new CPU designs, it lacks the capability for fine-grained predictive analysis of the thousands of individual modules within the CPU. GRANNITE [51] leverages Graph Neural Networks (GNNs) to perform transferable predictions of gate-level switching activity in combinational circuits. However, GRANNITE requires RTL simulation to obtain dynamic waveforms and necessitates logic synthesis to construct the connectivity graph of all constituent gates.

In this paper, leveraging large language models (LLMs), we propose BigPower, a model that enables fine-grained, module-level power consumption prediction directly at the source code level without the need for simulation. This allows us to identify power consumption bottlenecks in fine-grained modules at a lower time cost and with greater flexibility, thereby accelerating the cycle of design optimization.

*The corresponding author is Chunjie Luo (luochunjie@ict.ac.cn).

Our motivation is based on the observation that CPU power consumption is shaped by workload characteristics, microarchitectural configurations, and hierarchical module interactions. Existing approaches typically obtain such information through simulation-derived activity traces, which incur substantial cost. However, many relevant structural signals—including RTL implementation, module hierarchy, connectivity, architectural configurations, and workload descriptions—are already available at the source level. We hypothesize that learning from these heterogeneous signals can enable practical module-level power estimation without requiring additional simulation during inference. Large language models provide a flexible framework for encoding long-form structured source information and heterogeneous contextual representations, making them a promising foundation for this task.

To build BigPower, we first label a large module-level power dataset for training purposes. Utilizing seven different workloads, we conduct power labeling through EDA-flow (Electronic Design Automation) across 400 microarchitecture configurations of the open-source XiangShan processor family [46]. With approximately 5,000 module instances in XiangShan, the resulting module-level power labeling data amount to nearly 1.4×10^7 entries.

Next, we fine-tune our BigPower model based on an open-source pre-trained large language model. Constructing the input for the large language model is crucial. The input, denoted as $x = (c, w)$, comprise workload information (w) and a collection of contextual data (c) that represents CPU information. The corresponding output y represent the power consumption values of the module instances. The most significant challenge lays in representing the CPU information. We aim for the large model to comprehend both the overall CPU and the specific module instances requiring power prediction. However, due to GPU memory limiting, the input text length for large models and the complexity of the CPU code, it is impractical to input the entire CPU code into the model. Therefore, we propose a CPU information representation structure that combines global and local information, including microarchitecture configurations, CPU hierarchical structure information, module connectivity information, and module RTL code. The microarchitecture configurations comprise 20 microarchitecture parameter values which would be varied. The hierarchical structure of the CPU code could be viewed as a tree, and the hierarchical information we utilize included the path from the top-level module to the current instance, the sub-modules contained within the current module, and their power consumption values. The module connectivity information encompasses the names of all modules connected to the current module via signal lines. Given the varying lengths of different module RTL codes, we divide long code into several code segments, each constructing a separate input x . After constructing the (x, y) pairs, we employ these pairs to fine-tune the pre-trained open-source large model in a supervised manner.

Finally, we utilize ensemble learning and a bottom-up hierarchical approach to predict module-level power consumption. During prediction, we encounter two issues: 1) During fine-tuning, our hierarchical structure information include the power consumption of sub-modules. Therefore, we adopt a bottom-up prediction approach, first predicting the power consumption of bottom-level modules and then progressively predicting the power consumption of upper-level modules layer by layer. 2) Modules with long

Verilog code have multiple code segments, each constructing an x and yielding multiple prediction values. We average these multiple prediction values, which is an idea of ensemble learning. That also mitigates the randomness of large language model predictions.

We test the within-family generalization ability of BigPower in predicting power consumption under new CPU configurations, new workloads, and combinations of both. BigPower achieves $R^2 > 0.9$ prediction performance across these tests, demonstrating its capability to perform fine-grained, module-level power consumption predictions at the source code level.

Our contributions are as follows:

- We propose and implement fine-grained, module-level CPU power consumption prediction at the source code level without requiring simulation during inference, enabling us to identify power consumption bottlenecks in fine-grained modules at a lower time cost and with greater flexibility.
- We develop an LLM-based framework for CPU power consumption prediction, constructed a 14 million-entry power consumption dataset for training the LLM-based power model, and validate its effectiveness on the open-source XiangShan processor family.
- We introduce a CPU information representation structure that combines global and local information to address the challenge of directly inputting comprehensive CPU code information into large language models.
- We propose an ensemble learning and bottom-up hierarchical prediction method for module-level power consumption prediction.

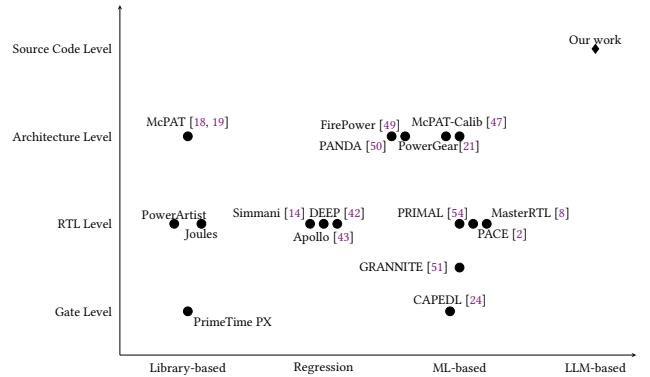


Figure 1: Power consumption analysis landscape. For clarity and simplicity, some related works have been omitted. Notably, Source Code Level implies that predictions are derived directly from static code properties, as opposed to features obtained from dynamic simulation.

2 Related Works

2.1 Power Analysis and Prediction

Analytical and library-based models: In order to get high accuracy and corporate with other EDA software, commercial tools for power analysis are usually based on power-characterized standard cell libraries and use analytical methods to produce stable and

traceable results. Synopsys PrimeTime PX (PTPX), which is one of the most commonly-used tools and often considered as some sort of golden standard, needs netlist and gate-level simulation waveforms to give precise and fine-grain power estimates. To avoid the time-consuming EDA flow to generate netlist and speed-up power feedback, other tools such as Ansys PowerArtist and Cadence Joules provide a coarse-grain power estimate using RTL codes and simulations. This rapid feedback capability is crucial for supporting agile hardware development methodologies.

As for academia, McPAT [18, 19] is the first to propose an integrated power, area, and timing modeling framework and has been constantly upgraded to adapt to new integrated circuit fabrication technology. McPAT-PVT [35] introduces a new FinFET design library to it and considers influence of process, supply voltage, and temperature (PVT) variations. McPAT-monolithic [12] enables modeling hybrid monolithic multi-core architectures. McPAT-7nm, as a part of McPAT-Calib [47] becomes an upgraded version adapted for 7nm technology.

Regression models: Compared to analytical methods, regression-based power models has been extensively researched since linear regression models and its numerous variations are easy to implement and have solid theoretic background. Researchers have developed numerous feature selection algorithms to improve their accuracy and to reduce overhead. Simmani [14] constructs a toggle-pattern for signals clustering and then selects these signals to serve as features. Using a minimax concave penalty (MCP)-based feature selection algorithm, Apollo [43] utilizes $\leq 0.05\%$ of RTL signals to train the power model using linear regression. DEEP [42] proposes a bit-level RTL signal selection and a two-step on-chip power meters selection method. The work [34] uses an interaction linear regression power model in order to guide register allocation. The literature [37] proposes that using CMOS MEMS magnetic field sensors combined with some commonly monitored system parameters, a multivariate nonlinear regression is capable to predict system power consumption well.

Machine learning model: Recent development of machine learning models enables capturing complex and intricate relationships in power consumption. Kumar et al. [15] build up component level power model, testing and comparing multiple ML regressors such as decision tree, gradient boosting model and random forest. In McPAT-Calib [47], XGBoost regressor serves to calibrate the output of McPAT. PANDA [50] develops its own simpler analytical function for each component and then uses XGBoost to handle more complex patterns. FirePower [49] introduces separate hardware and event models, both based on XGBoost, to adapt to few-shot learning scenario.

In recent years, neural networks have also been applied to this domain. PRIMAL [54] uses a convolutional neural network(CNN) to model cycle-by-cycle RTL power with detailed switching activities of all registers. GRANNITE [51] utilizes graph neural networks(GNNs) to predict gate-level toggle rates. Also, with GNN, PowerGear [21] predicts the power of FPGA in the high-level synthesis stage. MasterRTL [8] develops a new bit-level design representation and the power model part of it uses either a GNN or a lightweight tree-based model. It is worth noticing that an LLM-based data augmentation is introduced. The work [48] uses a multi-layer perceptron (MLP) with residual connection as a power model

and develops a cross-domain mix-up data generation for transfer learning. In CAPEDL [24], a two-step deep learning network is introduced, including an auto-encoder for signal compression and a MLP for ASIC designs' cycle-accurate power estimation. PACE [2] uses a sparse Multilayer Perceptron (sMLP) on the selected register bits to predict the power consumption of the CPU designs.

All works mentioned above need simulations to extract input features, whether architecture level [15, 21, 47–50], RTL level [2, 8, 51, 54] or gate level [24]. The requirement of simulations draws some disadvantage. Designs in the early stage are frequently incomplete or contain errors, rendering direct simulation infeasible without further refinement. Further more, relying on simulations introduces unnecessary correlation between different teams, because some specific and localized bugs might have negligible influence on other modules' power optimization.

2.2 Large Language Models

The Transformer architecture, introduced in 2017 [39], fundamentally shifted neural network design for natural language processing. By entirely replacing recurrence with a self-attention mechanism, it enabled superior parallel processing and long-range dependency handling, drastically improving training efficiency and model capacity. This led to the pervasive pre-training and fine-tuning paradigm. Subsequent models like GPT [30] and its successors (GPT-2 [31], GPT-3 [4]) advanced large-scale autoregressive pre-training, showcasing remarkable generation and pioneering in-context learning. Further, InstructGPT [28] focused on aligning LLMs with human preferences. More recently, the release of open-source models such as LLaMa [38] and Qwen [1] has broadened access to LLM technology, fostering wider research and development.

Large Language Models are increasingly being investigated for their capabilities within the domain of computer architecture. Current research demonstrates their utility in various aspects, including RTL code generation [5, 6, 23, 52], test bench creation and verification [13, 26], and even entire chip design [9, 41]. To support advancements in LLM-driven code generation, specialized benchmarks such as VerilogEval [22] and RTLML [25] have been established. Furthermore, LLMs are being actively adopted for High-Level Synthesis (HLS) code generation [10, 44] and optimization tasks [27]. LLMs also present capabilities in design space exploration [36, 40, 45].

In this paper, we utilize large language models to predict power consumption, enabling fine-grained, module-level power consumption prediction directly at the source code level without the need for simulation. Fig. 1 demonstrates the differences between our work and other works. Our proposed approach offers greater flexibility in power estimation, enabling rapid power analysis of arbitrary modules within new designs.

3 Methods

We frame the power estimation task as a supervised learning problem. The goal is to learn a function, f , that maps an input pair, $x = (c, w)$, to its corresponding power consumption, y . Here, c represents the comprehensive instance information, encompassing details such as its Verilog code, the CPU context it belongs to, and other relevant design parameters. w represents the software

workload. The model’s objective is to approximate this function:

$$y = f(x) = f(c, w) \quad (1)$$

Currently, there are numerous open-source pre-trained large language models available within the community. Adapting these general-purpose models to a specific vertical domain lies in the construction of a specialized dataset, which is then used to fine-tune the base model for domain-specific tasks. Therefore, for constructing BigPower, as illustrated in the Fig. 2, the critical steps are as follows:

- Labeling. Acquiring and assigning ground-truth values of power consumption y to certain instances of x .
- Dataset construction. The primary focus here is on how to represent $x = (c, w)$ and then pair it with y to form training pairs (x, y) .
- Fine-tuning. Utilize the (x, y) pairs to fine-tune the pre-trained large language model to adapt it to the power analysis domain.
- Predicting. Perform power consumption predictions for new workloads or new CPU modules.

Next, we will begin by introducing the acquisition of ground truth power consumption values y in Section 3.1. Subsequently, in Section 3.2, we will present the complete representation of $x = (c, w)$. Section 3.3 will introduce the fine-tuning process, and finally, Section 3.4 will explain how to conduct predictions.

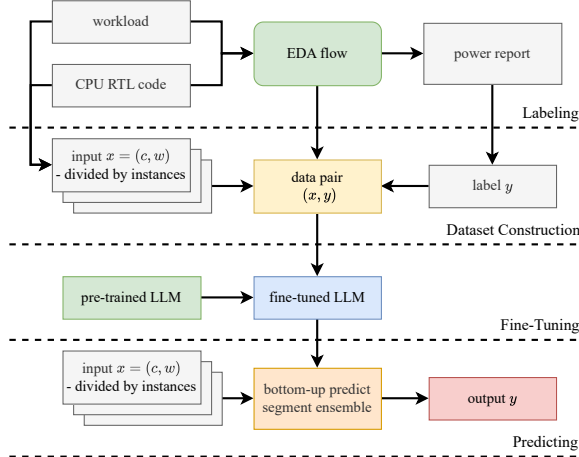


Figure 2: The overview of BigPower.

3.1 Labeling

We label the power consumption of different instances of open-source XiangShan processor family [46], by running various workloads on XiangShan with different microarchitecture configurations.

We use seven workloads which come from commonly used CPU benchmark suites. They include

- (1) hanoi: hanoi tower.
- (2) kmeans: k-means clustering algorithm.

- (3) qsort: quick sort algorithm.
- (4) multiply: multiplication implemented by bit-wise operation.
- (5) fib: matrix-multiplication-based fast power to calculate Fibonacci sequence.
- (6) mm_int: matrix multiplication of integer data type.
- (7) mm_float: matrix multiplication of single-floating-point data type.

The microarchitecture configurations are sampled from the space defined by Table 1, which encompasses approximately 5.8×10^9 possible configurations. We perform power labeling across nearly 400 microarchitecture configurations of the XiangShan. With approximately 5,000 module instances in XiangShan, the resulting module-level power labeling data amount to nearly $7 \times 400 \times 5000 = 1.4 \times 10^7$ entries.

Table 1: Microarchitecture configuration space.

Name	Candidate Value
ras_entry	8,16,24,32
decode_width	4,6
rob_entry	64,128,192,256,320
phy_register	64,128,192,256,320
int_multdiv	1,2,3,4
fp_misc	2,4,6,8
fp_mac	2,4,6,8
ldq_entry	32,48,64,80,96
stq_entry	32,48,64,80,96
icache_way	2,4,8
icache_tlb	16,32,48,64,80
icache_size (KB)	32,64,128
dcache_way	2,4,8
dcache_tlb	32,48,64,80,96
dcache_size (KB)	32,64,128
dcache_mshr	16,32,64
l2_size (MB)	1,2,4
l3_size (MB)	2,4,8,16,32
l2_assoc	4,8,16
l3_assoc	4,8,16

3.2 Dataset Construction

Large language models are based on Transformer [39] architecture. The attention mechanism of traditional Transformers exhibits a quadratic time complexity ($O(n^2)$) with respect to the input sequence length n , posing inherent challenges for long context processing. Although modern LLMs have seen advancements in extending context windows, the practical deployment of very large context lengths remains constrained by our limited memory and computational resource budgets. Therefore, it is necessary to partition the comprehensive power estimation task for an entire CPU design into distinct instances. The question is how.

3.2.1 Overview. Consider a hypothetical scenario: a fine-tuned LLM achieves power consumption prediction accuracy comparable to commercial EDA tools. This thought experiment prompts us to

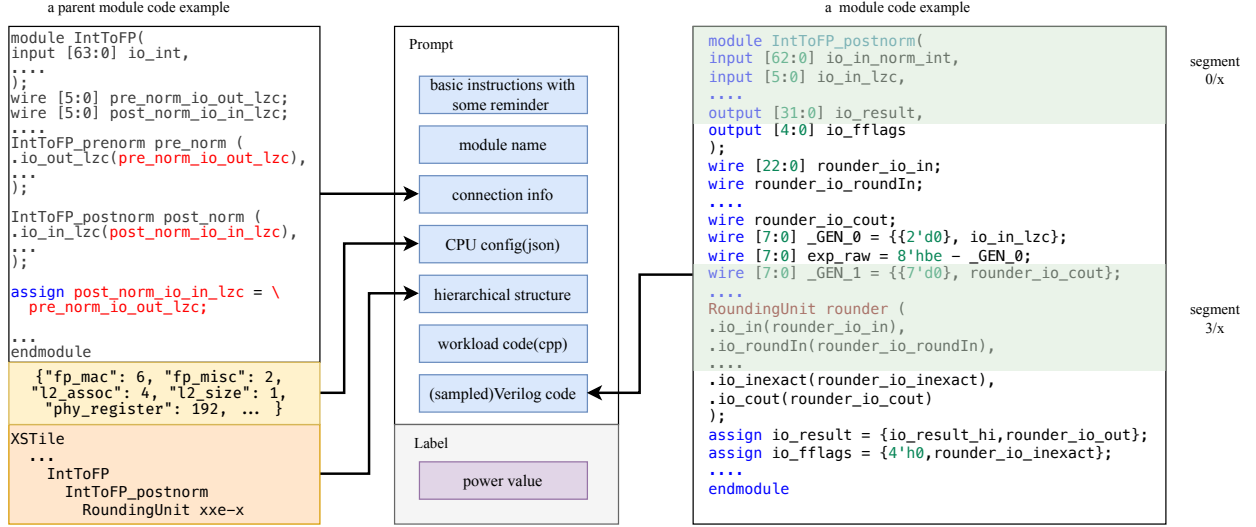


Figure 3: Comprehensive illustration of data pair construction for power prediction. The figure provides a detailed view of the input x (comprising a prompt structure and associated data) and the output y (power value). The central panel illustrates the composition of the LLM prompt. The right panel shows a `IntToFP_postnorm` module’s partial Verilog code, highlighting how it is divided into segments for input. The upper-left panel depicts a parent module `IntToFP` and its two interconnected child module instances (`IntToFP_prenorm`, `IntToFP_postnorm`), while the middle-left panel presents CPU configuration parameters in JSON format. The bottom-left panel depicts the hierarchical structure of the `IntToFP_postnorm` module.

investigate the theoretical minimum information required for such an achievement.

Traditional commercial EDA flow typically need standard cell libraries, RTL code or netlist and simulation waveforms. When developing a CPU within a specific technology node, standard cell libraries can be considered static and thus learned as background knowledge by the model. Our work focuses on front-end factors affecting power consumption, which are independent of the later-stage physical implementation process. To isolate our analysis, we assume a constant, high-quality physical implementation, effectively neutralizing the impact of these typical back-end factors.

However, the power consumption of an individual module instance is intrinsically linked not only to the instance’s own characteristics but also to the broader context of the entire CPU design and its interactions with other modules. Consequently, the isolated Verilog definition of an instance (as discussed in Section 3.2.4) is insufficient. To address this, connection information (detailed in Section 3.2.2) and the hierarchical structure of the design (elaborated in Section 3.2.3) are crucial inputs. Furthermore, for a given design, simulation waveforms are determined by the applied workload, a topic explored in Section 3.2.5.

In summary, the complete input pair, $x = (c, w)$, is illustrated in Fig. 3. These processing steps are minor, typically completing within a few minutes. Our method requires only partial information from the RTL design and thus is much faster than compilation and elaboration step necessary for conventional simulation, which must parse and comprehend the entire design’s semantic and behavioral logic to generate executable code.

3.2.2 Connection Information. The upper left side of Fig. 3 shows an example that one parent module `IntToFP` has two instantiated child modules `IntToFP_prenorm` and `IntToFP_postnorm`. It is worth noticing that there are two red-colored signals, `pre_norm_io_out_lzc` and `post_norm_io_in_lzc`, mapped to child modules’ input or output ports and they have direct connection to each other by the blue-colored Verilog keyword `assign`. The `assign` statement in Verilog facilitates direct continuous assignment compared with procedural assignment. We consider that any two instances have interaction with each other only if they have common signals identified by `assign` statement. A Python binding of slang (Pyslang) [29], which is a fast and compliant SystemVerilog frontend, is used to extract those relationship. This information will be cached to speed up dataset construction and later be represented in a natural language tongue.

3.2.3 Hierarchical Structure. The right side of Fig. 3 is a demonstration of Verilog code of a particular module. Verilog code is organized in such a way that child modules are instantiated as components within a higher-level parent module, forming a hierarchical tree structure of instances as illustrated in Fig. 4. We consider the hierarchical structure is an important part of the instance information c . More specifically, all the ancestor instances of the instances form a path and every direct child instances with their corresponding power consumption values are listed, which is represented in an indent-based tree form as show in the left part of Fig. 3. Note that we only provide the module names because including the code of all these modules as input would result in an excessively long length.

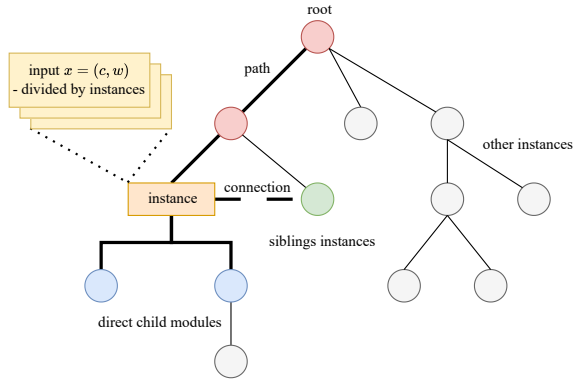


Figure 4: Hierarchical tree representation of a CPU design. Each node (circles and rectangle) signifies a module instance. An example instance (orange) is highlighted, showing how the input $x = (c, w)$ is constructed per-instance. The diagram also depicts hierarchical relationships (e.g., 'root', 'path', 'direct child modules') and dotted-line inter-instance 'connection' (between siblings).

3.2.4 Segmented Verilog Code. Using the full Verilog code for certain instances might pose a challenge due to excessive code length. To address this, we truncate overly long code into multiple segments, as illustrated by the green shaded areas on the right side of Fig. 3.

Segments, while only representing a portion of the original code, are randomly sampled and concatenated to construct the input $x = (c, w)$. The key distinction among these inputs is the specific code segment, while all other contextual information and the corresponding power consumption remain constant. In machine learning terms, this approach is analogous to data masking, where the model is exposed to only partial information yet is trained to robustly and accurately predict the output. Notably, the sampled Verilog code segments are positioned at the end of the input prompt. When the overall input length necessitates truncation, the Verilog code information is the first to be reduced, minimizing its impact on the model's predictive accuracy.

Modules lacking child modules, often referred to as leaf modules, typically possess concise Verilog code and are commonly instantiated multiple times across a design. We argue that these modules are typically too fine-grained and, individually, contribute negligibly to overall power consumption, thus attracting minimal designer attention. Consequently, leaf modules with code shorter than a defined threshold are treated as an integral part of their respective parent modules. Their Verilog code is then concatenated with that of their respective parent modules before segmentation.

3.2.5 Workload and Other Parts. Currently, our power prediction methodology utilizes simple C++ programs as workloads, with their code manually selected to serve as input for the LLM. The extension to handling larger or more diverse programs is reserved for future research.

We employ a unified instruction format to prompt the LLM for power prediction. The prompt explicitly directs the LLM to sum the power consumption of all child modules' power consumption and to ensure the predicted value is not less than the sum. To provide the LLM with an understanding of the overall CPU context, microarchitecture parameters are also included as part of the input $x = (c, w)$ in JSON format, depicted on the middle-left side of Fig. 3.

Finally, inspired by the work of Gruver et al. [11], we introduce an encoding method for floating-point power numbers to enhance the LLM's numerical stability. Specifically, decimal points are omitted by fixing the precision, transforming numbers like $1.234e-3$ to $1234e-6$.

3.3 Fine-tuning

We choose a pre-trained open source model, LLaMa 3.1 8B-Instruct [7] as the foundation model of BigPower, which is a famous light weight model and is well supported by the society. Our fine-tuning process is implemented using the LLaMa-Factory [53].

We use the supervised fine-tuning and detailed hyper-parameters setup will be discussed in experiments setup Section 4.

To uniquely identify power prediction results for different instances, we incorporated an id property as meta-information within our dataset. To accommodate this, a minor adaptation is made to LLaMA-Factory's output handling, specifically to bypass the default processing of this id meta-property. This modification solely pertains to metadata handling and does not alter LLaMA-Factory's fundamental training or inference processes.

3.4 Predicting

The input x for a given instance requires knowledge of the power consumption of its child modules. During the training phase, the ground truth power consumption for all modules (both parent and child) is known. This allows us to circumvent the inherent inter-module dependencies during training, as the model has direct access to the required child module power values.

For the prediction process, we propose a hierarchical "bottom-up" approach. In this methodology, module instances are systematically classified according to their level within the CPU's hierarchical tree structure (illustrated in Fig. 4). Power consumption estimates are then progressively computed, starting from the lowest-level modules (the "bottom") and proceeding upward to their respective parents, until the entire CPU's power is estimated.

Numerical stability is a concern when employing LLMs for regression tasks, particularly when dealing with floating-point predictions. It is clear that the digits in the exponent portion of a floating-point number influence its magnitude and, consequently, the prediction's overall accuracy, far more than the fractional part. To enhance the robustness of our LLM's predictions, especially given the inherent challenges of directly predicting precise floating-point values, we implement several strategies.

Ensemble prediction via segmentation: As detailed in Section 3.2.4, a single long Verilog code from instance is segmented, generating multiple distinct inputs for the LLM. Each segment yields an power consumption prediction for the same instance. It should be noted that during training, the ground truth labels correspond

to the power of the entire module rather than that of individual segments, as per-segment power data is unavailable. Consequently, every segment within the same module is trained to predict the total module power, essentially estimating the whole from various constrained perspectives. To mitigate variance and improve stability, the median of these multiple predictions is then taken as an ensemble result. This is inspired by ensemble learning, which is widely applied in machine learning. In this approach, each learner makes predictions about the whole based on partial features, and then the predictions from all learners are integrated (through voting for classification or averaging for regression).

Outlier handling and magnitude correction: We also integrate a mechanism for addressing anomalous predictions. During the prediction process, we calculate the average power consumption and its standard deviation across all modules in the training set. It has been observed that the LLM occasionally predicts power values with significant errors in the exponent (magnitude), particularly when the estimation deviates substantially from the historical average. To counter these numerical instabilities, such outlier predictions are identified and subsequently replaced with the calculated average power consumption.

4 Experiment Settings

4.1 EDA-flow

Our work utilizes the XiangShan Nanhu version of the CPU design. All Verilog code is generated using the official Chisel framework, and all comments and debug information are systematically removed to ensure a clean input for subsequent processing (e.g., LLM ingestion). The gate-level netlist is then synthesized using Synopsys Design Compiler, while simulations are performed with Synopsys VCS. Finally, detailed power reports are generated using Synopsys PrimeTime PX. We have labeled the power consumption for 400 CPU configurations. Due to hardware resource limitations, fine-tuning with the entire dataset would be excessively time-consuming. Therefore, in this paper, we did not use the full dataset when validating our method. We randomly select 20 configurations as training data and 4 as test data in our experiments. We plan to open-source the complete labeled dataset.

4.2 Model tuning

The experimental setup comprise four NVIDIA L40S GPUs, each with 48GB of ECC memory. During the fine-tuning of the LLM, a learning rate of 2×10^{-6} is used with a cosine scheduler and no warmup steps. Fine-tuning is limited to a single epoch on the full training dataset. To mitigate memory and computational resource limitations, BFloat16 (BF16) quantization is employed. We used a global batch size of 4 (one sample per GPU) with 32 gradient accumulation steps. During fine-tuning, a partial training strategy is adopted where only the last 10 layers of the LLM are updated, while all remaining trainable modules are frozen, including the default tokenizer and embedding layer of original LLaMA 3.1. For efficient memory management and accelerated training, we leverage DeepSpeed’s [33] Zero Redundancy Optimizer (ZeRO) Stage 2 [32] with CPU offloading. The maximum sequence length is set at 8192 tokens. Sequences exceeding this limit will be truncated via the framework’s dynamic length allocation strategy. Model

hyper-parameters are chosen based on common settings found in similar LLM fine-tuning tasks and not subjected to comprehensive optimization, given our resource constraints.

4.3 Metrics

Power modeling constitutes a regression problem. In evaluating the accuracy of modeling outcomes, we employ two commonly adopted metrics: the Mean Absolute Percentage Error (MAPE) and the Coefficient of Determination (R^2).

$$\text{MAPE} = \frac{1}{n} \sum_i^n \left| \frac{\hat{y}_i - y_i}{y_i} \right| \times 100\% \quad (2)$$

$$R^2 = 1 - \frac{\sum_i^n (\hat{y}_i - y_i)^2}{\sum_i^n (y_i - \bar{y})^2} \quad (3)$$

where y_i is the ground truth, $\hat{y}_i$ is the prediction, and $\bar{y}$ is the average of ground truth derived from the commercial EDA-flow detailed in in Section 4.1. MAPE is calculated based on the normalized difference (often expressed as a percentage) between the actual power values and their corresponding predicted values. A lower MAPE signifies a smaller modeling error, indicating more accurate predictions. R^2 measures the proportion of the variance in the dependent variable that is explainable by the independent variables within a regression model. A high R^2 value denotes a strong correlation between the variables. The maximum value of R^2 is 1, which indicates that the predictive model perfectly matches the true values.

5 Main Results

In this section, we test the generalization ability of BigPower and compare it with traditional models.

5.1 Generalization Across Configurations and Workloads

To evaluate the performance on unseen microarchitecture configurations and workloads, we test the generalization ability of BigPower under new CPU configurations, new workloads, and combinations of both. The results, comprising MAPE and R^2 scores, are summarized in Fig. 5.

5.1.1 New Configurations, Known Workloads. First, we evaluate the model on four unseen configurations using the seven workloads included in the fine-tuning set. As shown in Fig. 5(a), the model achieves an average MAPE of 12.09% and a strong R^2 of 0.97. While performance varies by workload ranging from 3.02% to 25.36%, the consistently high R^2 values (0.91–0.99) indicate that BigPower effectively captures the power scaling trends across different hardware parameters.

5.1.2 Known Configurations, New Workloads. Second, we test the model’s capability on seven entirely new workloads across known configurations.

- (1) bf: Brainfuck interpreter.
- (2) dhrystone: A synthetic benchmark designed to measure integer arithmetic and string manipulation performance.
- (3) lzip: Data compression utility that employs the Lempel-Ziv-Markov chain algorithm (LZMA)

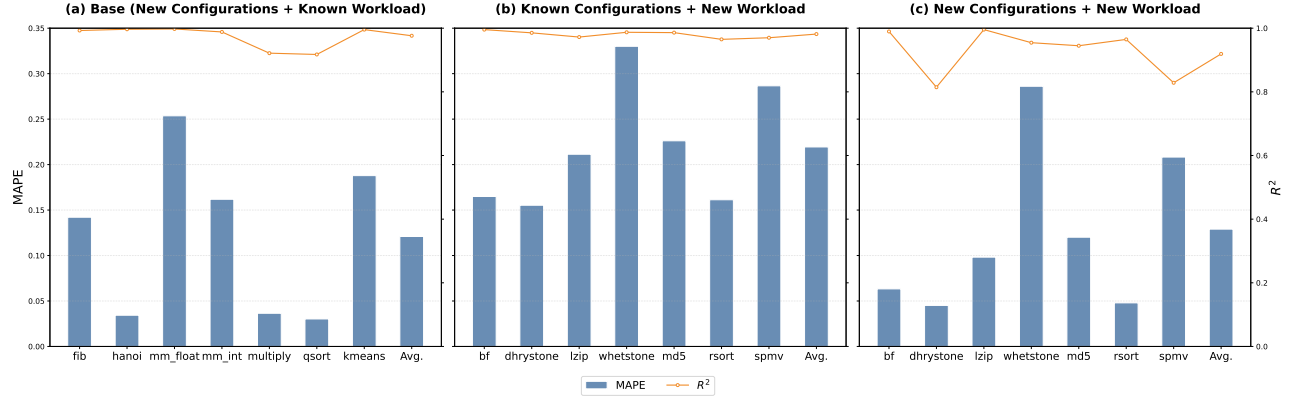


Figure 5: Validation of BigPower. Comparison of MAPE (bars) and R^2 (lines) across varying configurations and workloads. Sub-labels on the X-axis denote specific benchmarks, with "Avg." representing the mean performance across the test.

- (4) whetstone: A synthetic benchmark focused on floating-point arithmetic operations.
- (5) md5: An implementation of the MD5 cryptographic hash function
- (6) rsort: Radix sort algorithm.
- (7) spmv: Sparse Matrix-Vector Multiplication.

As shown in Fig. 5(b), the average MAPE increases to 21.94%, with an average R^2 of 0.94. This increase in error is likely attributable to the relatively sparse diversity of the training workload.

5.1.3 New Configurations and New Workloads. Finally, we consider the most challenging scenario: predicting power for new workloads on new configurations. As shown in Fig. 5(c), BigPower achieves a surprisingly average MAPE of 12.90% and an average R^2 of 0.92. Although the average error remains low, we observe a notable decline in R^2 for specific benchmarks, such as whetstone and spmv. The decline in R^2 indicates a diminished capacity to track underlying trends compared to the previous two scenarios.

Nevertheless, in all three evaluation settings, BigPower achieves an overall average MAPE of 15.64% and R^2 of 0.94, demonstrating its effectiveness in fine-grained module power prediction without the need for simulation.

5.2 Comparison with Traditional Models

It is difficult to perform direct comparisons with other power models across different target designs and application scenarios. To achieve the fine-grained, module-level power prediction demonstrated in our work, frameworks such as Primal [54] and Apollo [43] would require constructing and training individual models for each of the nearly 5,000 module instances within the XiangShan processor. Furthermore, Primal and Apollo operate at the RTL level. While GRANNITE [51] offers transferable predictions, it necessitates both RTL simulation and logic synthesis to get the dynamic toggle rates of the registers and the graph representation of the gate-level netlist. For large-scale circuits like XiangShan, the resulting graph size becomes prohibitively massive.

In prior works [16, 17], design parameters are used to perform regression modeling for design space exploration. Because these

methods also do not utilize event characteristics obtained through dynamic simulation, they are unable to model varying benchmark behaviors and lack the capability for fine-grained power prediction.

To enable fine-grained power consumption prediction by traditional regression models, avoiding the need for features generated from dynamic simulations, we adopt a one-hot encoding scheme for both workloads and modules. The resultant input vectors consist of these one-hot encodings alongside system configuration parameters. A key challenge in encoding modules within the XiangShan architecture is that a single module code can correspond to a variable number of instantiated physical instances depending on the configuration. For the purpose of our encoding, all modules with the identical name are grouped and treated as the same unit. To account for this variability, our approach involves one-hot encoding the module's name only, with the prediction targeting the average power consumption across all its corresponding instances.

We experiment with different types of commonly used regression models, including Ridge, Lasso, SVR, Random forest, Gradient boost, Neural network.

Table 2 presents the MAPE values for these traditional regression models and our BigPower. As can be seen, the two linear regression models perform extremely poorly on this task. While SVR, Random forest, Gradient boost, and Neural network perform slightly better than linear regression, their average MAPE values still range from 182.22% to 449.62%. Only our BigPower achieves satisfactory prediction results.

6 Ablation Study

This section presents several ablation experiments to verify the effectiveness of the strategies employed during data construction and the prediction process.

6.1 Ablation Study for Dataset Construction

We conduct preliminary ablation studies to determine the optimal strategy for constructing the training dataset. For rapid iteration, several experiments utilize a simplified setup. The test set contains randomly sampled 7×10^5 pairs, with 1×10^5 pairs drawn from each of the seven distinct workloads. The test dataset use CPU

Table 2: Comparison of MAPE (%) with different models.

	Ridge	Lasso	SVR	Random Forest	Gradient Boost	Neural Network	Ours
fib	11334.31	73695.22	206.22	278.49	558.07	418.66	14.21
hanoi	6111.65	52328.31	130.17	194.79	308.77	339.22	3.43
mm_float	9398.31	73904.33	185.94	280.59	465.54	425.52	25.36
mm_int	11588.88	74227.57	238.73	284.36	548.95	403.95	16.18
multiply	8009.40	69553.33	170.59	262.65	498.59	420.13	3.64
qsort	9211.21	67538.09	166.66	256.29	533.02	478.78	3.02
kmeans	5362.24	45925.41	177.22	170.55	234.42	343.69	18.79
Average	8716.57	65310.32	182.22	246.82	449.62	404.28	12.09

configurations that are unseen during training. For the ablation study, test and training datasets are generated and evaluated pairwise, in order to explore the effects of different dataset construction methodologies. Notably, the ground-truth power values of child modules are provided directly in the input prompt for this ablation study.

6.1.1 Impact of Dataset Size. We first evaluate the correlation between training data volume and model performance. As illustrated in Fig. 6(a), prediction accuracy improves consistently as the dataset scales. The model trained on the "full" dataset significantly outperforms smaller subsets, achieving the lowest MAPE. The evaluated configurations are defined as follows:

- (1) small: A random sample of 7×10^5 input pairs.
- (2) medium: A larger set of 7×10^6 pairs, sampled evenly from 7 distinct workloads.
- (3) large: An even larger set of 1.4×10^7 pairs, sampled evenly from 7 distinct workloads.
- (4) full: One complete training epoch on the entire dataset.

6.1.2 Impact of Prompt Composition. We further investigate the contribution of individual components within the LLM prompt. These experiments utilize the medium dataset to balance training efficiency with performance insights. The results are summarized in Fig. 6(b). We examine the following variations:

- (1) w/o connection info: Omits explicit information about sibling module connectivity.
- (2) w/o reminder: Removes instructional cues and reminders from the prompt.
- (3) w/o code: Excludes Verilog code while retaining all other prompt elements.
- (4) w/o hier tree: Removes the hierarchical tree structure

Using the medium configuration as the baseline (indicated by the red dashed line), most ablation variants exhibit inferior performance (higher MAPE). A notable exception is the w/o code variant, which appears to outperform the baseline in the simplified setup. We suppose that Verilog code, as a form of unstructured text, is intrinsically information-sparse. Consequently, the model can only effectively leverage such information when supported by a sufficiently large training dataset. To investigate this further, we conducted a full-scale training run without code, evaluated using the final bottom-up paradigm detailed in Section 3.4. As shown in the right-hand portion of Fig. 6(b), our proposed method (with

code) ultimately achieves a lower MAPE when scaled to the full dataset, justifying its inclusion in the final architecture.

To further investigate the impact of Verilog code on model performance, we characterized the code length distribution across different modules in XiangShan. As illustrated in Fig. 7(a), the majority of instances in XiangShan are relatively small in terms of the Verilog code length. Intuitively, while the power characteristics of simple modules may be easily captured via structural information, complex modules described by extensive code segments are likely to derive greater benefit from our code segment strategy.

To test this, we isolated a subset of the "largest" modules (the top 10% by code length) to compare model performance. As shown in Fig. 7(b), our proposed method achieves a robustly low MAPE across both the full test set and the big-module subset. In contrast, the performance of the "without code" variant degrades significantly when restricted to these complex modules, experiencing a sharp increase in MAPE. This confirms that the inclusion of Verilog code is critical for modeling the power consumption.

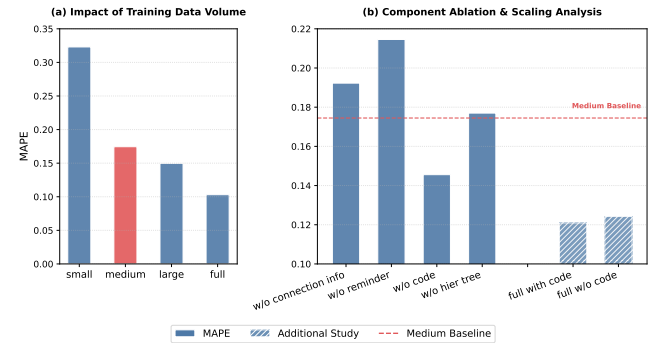


Figure 6: Evaluation of dataset construction strategies. (a) Impact of training data scale on model accuracy, showing a consistent reduction in MAPE as the dataset scales. (b) Ablation of prompt components on the medium dataset (left), followed by a comparative validation of the Verilog code impact at full scale (right). The red dashed line represents the baseline performance of the standard medium dataset configuration.

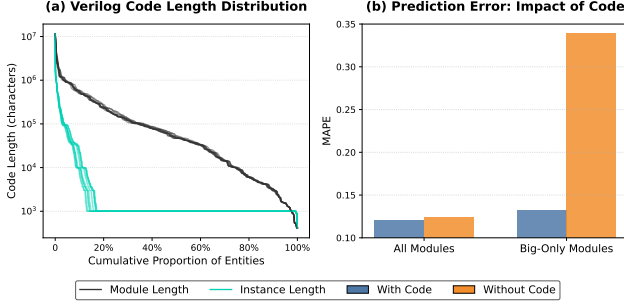


Figure 7: Analysis of Verilog code complexity. (a) Cumulative distribution of Verilog code length (after cleaning) for modules and their corresponding instances. Each line represents a unique CPU configuration. (b) Comparison of prediction error between the full proposed model and the "without code" variant.

6.2 Ablation Study for Predicting

We conduct an ablation study to validate the effectiveness of the bottom-up prediction strategy detailed in Section 3.4. As discussed in Section 3.2.4, instances with Verilog code exceeding the model’s context length are segmented, necessitating an ensemble strategy to consolidate their multiple prediction outputs. Given that our recursive bottom-up approach has the potential to allow errors to propagate through the hierarchy, the choice of ensemble method is critical. The following configurations are evaluated:

- (1) Base (Proposed Method): The standard recursive approach. For segmented instances, the median of all segment predictions is used as the module’s power value. This value then serves as the child-module input for the parent’s power prediction.
- (2) w/o bottom-up: The recursive approach is disabled; power values for all child modules are fixed at default value of zero in the input as a placeholder for missing hierarchical information.
- (3) First Element: A baseline that uses only the prediction from the first code segment.
- (4) Random: A baseline that randomly selects the prediction from one of the segments.

As shown in Fig. 8, the Base method achieves the lowest MAPE. The w/o bottom-up variant is inferior to all recursive approaches. We attribute this partially to the model encountering out-of-distribution (OOD) data; as the LLM is not trained on inputs where all child modules have zero power, leading to poor generalization in this scenario. Furthermore, the First Element and Random methods are inherently arbitrary and exhibit less numerical stability than our median-based approach, resulting in higher overall error.

7 Case Study

In this section, we show the practical advantages of BigPower through two use cases. 1) Hierarchical power breakdown: our ability to perform fine-grained, module-level predictions allows for a layer-by-layer diagnostic of specific designs. 2) Design change

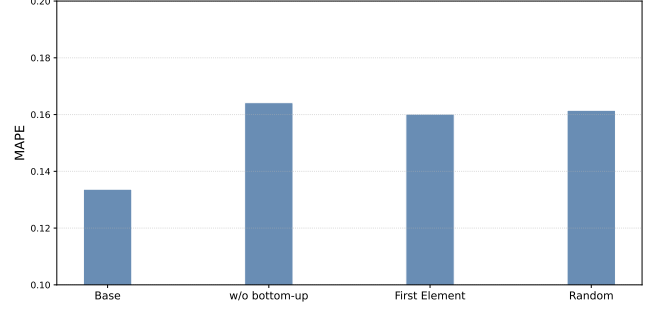


Figure 8: Impact of hierarchical prediction and segment ensemble.

sensitivity analysis: we can clearly contrast power fluctuations resulting from architectural modifications. Crucially, these applications eliminate the requirement for simulation. By leveraging sufficient GPU resources, BigPower could enable massive parallel prediction across modules, thereby accelerating the design iteration cycle.

7.1 Hierarchical Power Breakdown

In contrast to most existing works, BigPower predicts power consumption directly at the source code level, facilitating a comprehensive top-down analysis. This granularity enables researchers to pinpoint and optimize power hot spots. Fig. 9 illustrates the prediction error distribution across the architectural hierarchy with a specific configuration (Configuration 1, C_1) on the workload Hanoi.

We define the relative error as:

$$\Delta_p = \frac{P_{\text{module_predict}}}{P_{\text{root_predict}}} - \frac{P_{\text{module_label}}}{P_{\text{root_label}}} \quad (4)$$

This metric characterizes the model’s accuracy in identifying the relative power proportion of each component. In the visualization, the color intensity represents the error magnitude, where red hues indicate an overestimation of a module’s power share and blue hues indicate an underestimation.

Using the XScore as the root module, the visualization reveals a consistent "near-white" color profile across all hierarchical levels. This signifies that BigPower maintains high fidelity in preserving spatial power locality, ensuring that the predicted power profile closely mirrors the ground-truth distribution across functional units. Minor deviations occur in Scheduler_1 and ExuBlock_1, where the model slightly overestimates power contributions by +2.50% and +2.76%, respectively. Notably, ExuBlock_1 serves as the floating-point unit in the XiangShan architecture. Although the selected workload (Hanoi) is primarily a fixed-point program, the early-stage XiangShan design features relatively immature clock gating. Consequently, BigPower correctly captures a significant power share in modules that would otherwise be expected to remain idle or "dimmed" in a more power-optimized implementation.

7.2 Design Change Sensitivity Analysis

To demonstrate the diagnostic utility of BigPower, we perform a comparative analysis between two distinct processor configurations.

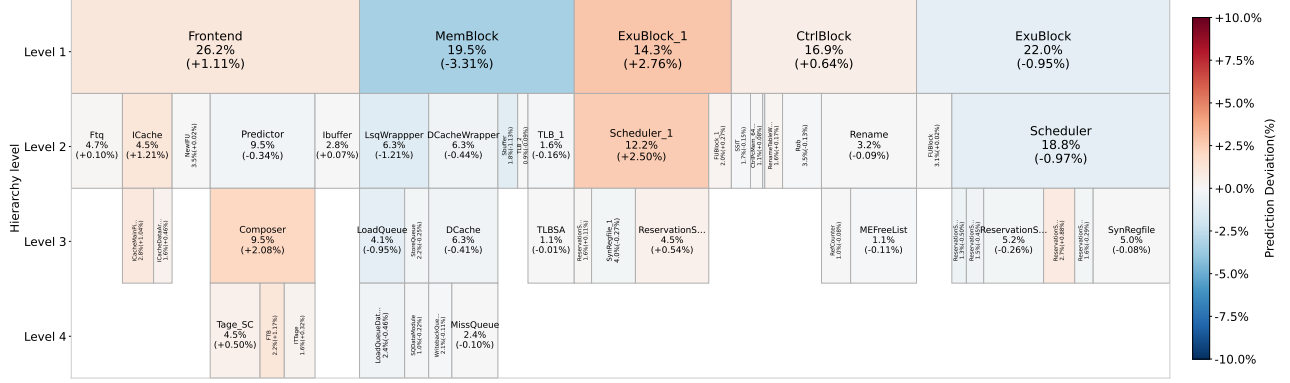


Figure 9: Hierarchical power breakdown. For visual clarity, the root and negligible sub-modules are omitted. Each block displays the module name, its predicted power share in current hierarchical level, and relative error Δ_P (in brackets). Color intensity represents prediction error, with red (+) for overestimation and blue (-) for underestimation.

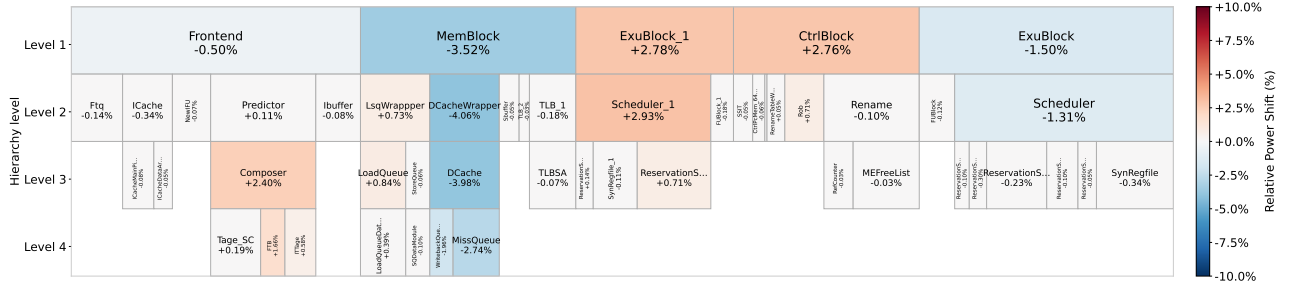


Figure 10: Design change sensitivity analysis. Each block displays a module’s name and its relative power shift (Δ_{shift}) between C_1 and C_2 . Color intensity reflects the magnitude of change: red (+) represents an increased power footprint in C_1 , while blue (-) represents a decrease.

Fig. 10 provides a fine-grained view of how architectural parameter tuning impacts the core’s power profile.

In this case, we define the predicted power shift (Δ_{shift}) as the difference in the predicted power proportion of a module between Configuration 1 (C_1) and Configuration 2 (C_2):

$$\Delta_{shift} = \left(\frac{P_{module_predict}}{P_{root_predict}} \right)_{C_1} - \left(\frac{P_{module_predict}}{P_{root_predict}} \right)_{C_2} \quad (5)$$

The parameter variations between C_1 and C_2 are summarized in Table 3. Notably, C_1 features fewer Miss Status Holding Register (MSHR) entries than C_2 . This architectural reduction is apparent in Fig. 10, where the DCache and DCacheWrapper blocks are highlighted with deep blue shading. Conversely, C_1 employs a larger LoadQueue (64 entries) compared to C_2 (48 entries). BigPower captures this overhead, represented by the light red tint in the LoadQueue block. Our results indicate that TLB entry count has a negligible impact on power distribution for this specific workload. It should be noted that the L3 Cache is excluded from Fig. 10, as the visualization is restricted to the power distribution within the XSCore. Furthermore, configuration changes may indirectly affect certain modules, e.g. ExuBlock_1 and CtrlBlock.

Table 3: Parameter variations in configurations C_1 and C_2 .

	ldq_entry	dcache_tlb	dcache_mshr	l3_size (MB)
C_1	64	32	32	16
C_2	48	64	64	32

8 Limitations

Although we have effectively predicted power consumption using LLMs, there remain certain limitations. First, cycle-accurate power prediction remains challenging. Predicting per-cycle power consumption without dynamic simulation is an extremely challenging task. It requires predicting power fluctuations from the start to the end of program execution. Second, we do not account for back-end physical implementation. Consistent with most power models, our current approach focuses on evaluating the power performance of CPU front-end of power flow. Third, the complexity of workloads is currently limited. Labeling power consumption using PTPX is time-consuming, making it nearly impossible for complex workloads. This prevents us from obtaining true power consumption values for these complex workloads through simulation, a challenge faced by power modeling research.

9 Conclusion

This paper presents BigPower, an LLM-based hierarchical framework for fine-grained module-level power estimation from source-level CPU design information. By leveraging workload characteristics, architectural hierarchy, module connectivity, configuration parameters, and RTL code representations, BigPower estimates module-level power consumption without requiring additional cycle-level simulation during inference. BigPower is trained on an extensive module-level labeled dataset spanning diverse workloads and CPU configurations of the XiangShan processor family. To address hierarchical dependency and long-context challenges, BigPower employs a structured representation combining global and local CPU context together with a bottom-up hierarchical inference strategy and ensemble aggregation. Experimental results demonstrate practical within-family generalization across unseen CPU configurations and workloads in XiangShan. Overall, BigPower provides an efficient and practical framework for fine-grained module-level power estimation during CPU development.

While BigPower currently focuses on within-family module-level estimation and does not model physical back-end effects, our findings suggest that source-level representations combined with hierarchical architectural context provide a promising direction for practical CPU power estimation. Future work includes extending workload diversity, improving cross-architecture transferability, and exploring finer temporal granularity.

References

- [1] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenhang Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, K. Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Yu Bowen, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xing Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. *ArXiv abs/2309.16609* (2023).
- [2] Cem Benar, George Phan, Sylvia Chan, Zongfang Lin, Yat Fai Lam, and Robert Chu. 2024. PACE: MLP-Based Fast and Accurate Per-Cycle Chip Power Modelling. In *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, Knoxville, TN, USA, 689–693. <https://doi.org/10.1109/ISVLSI61997.2024.00132>
- [3] David Brooks, Vivek Tiwari, and Margaret Martonosi. 2000. Wattch: A framework for architectural-level power analysis and optimizations. *ACM SIGARCH Computer Architecture News* 28, 2 (2000), 83–94.
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. J. Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Ma teusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *ArXiv abs/2005.14165* (2020).
- [5] Kaiyan Chang, Kun Wang, Nan Yang, Ying Wang, Dantong Jin, Wenlong Zhu, Zhirong Chen, Cangyuan Li, Hao Yan, Yunhao Zhou, Zhuoliang Zhao, Yuan Cheng, Yudong Pan, Yiqi Liu, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. 2024. Data is all you need: Finetuning LLMs for Chip Design via an Automated design-data augmentation framework. *Proceedings of the 61st ACM/IEEE Design Automation Conference* (2024).
- [6] Zhirong Chen, Kaiyan Chang, Zhuolin Li, Xinyang He, Chujie Chen, Cangyuan Li, Mengdi Wang, Haobo Xu, Yinhe Han, and Ying Wang. 2025. ChipSeek-R1: Generating Human-Surpassing RTL with LLM via Hierarchical Reward-Driven Reinforcement Learning. *arXiv preprint arXiv:2507.04736* (2025).
- [7] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony S. Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aur'elien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Rozière, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cris tian Cantón Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab A. AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Frank Zhang, Gabriele Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Graeme Nail, Grégoire Mialon, Guanglong Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel M. Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmur van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Ju-Qing Jia, Kalyan Vasuden Alwala, K. Upasani, Kate Plawiak, Keqian Li, Ken-591 neth Heafield, Kevin R. Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuen ley Chiu, Kunal Bhatta, Lauren Rantala-Yearry, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melissa Hall Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Niko lay Bashlykov, Nikolay Bogoychev, Niladri S. Chatterji, Olivier Duchenne, Onur cCelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasić, Peter Weng, Prajwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohit Girdhar, Rohit Patel, Ro main Sauvestre, Ron nie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sa hana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoqiang Nie, Sharan Narang, Sharath Chandar Parapathy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stéphane Collet, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkze, Vincent Gouget, Vir ginie Do, Vish Vogeti, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenxin Fu, Whit ney Meers, Xavier Martinet, Xiaodong Wang, Xiaoqing Ellen Tan, Xinfeng Xie, Xuchao Jia, Xuwei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yiqian Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zhengxu Yan, Zhengxing Chen, Zoe Papakipos, Aaditya K. Singh, Aaron Grattafiori, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adi Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alex Vaughan, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Franco, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardt, Po-Yao (Bernie) Huang, Beth Loyd, Beto de Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christopher Feichtenhofer, Da mon Civin, Dana Beaty, Daniel Kreymer, Shang-Wen Li, Danny Wyatt, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuel, Feng Tian, Firat Ozgenel, Francesco Caggioni, Francisco Guzm'an, Frank J. Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Govind Thattai, Grant Herman, Grigory G. Sizov, Guangyi Zhang, Guna Lakshminarayanan, Hamid Shojanazeri, Han Zou, Hannah Wang, Han Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Igor Molybog, Igor Tufanov, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kaixing(Kai) Wu, U KamHou, Karan Saxena, Karthik Prasad, Kartikay Khandelwal, Katayoun Zand, Kathy Matsosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kun Huang, Kunal Chawla, Kunal Lakhotia, Kyle Huang, Lailin Chen, Lakshya Garg, A Lavender, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabza, Manav Avalani, Manish Bhatt, Maria Tsimpoukeli, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navy ata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikolay Pavlovich Laptev, Ning Dong, Ning Zhang, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pe dro Ritner, Philip Bontrager, Pierre Roux, Piotr Dollár, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Raymond Li, Rebekkah Hogan, Robin Battley, Rocky Wang, Rohan Maheswari, Russ Howes, Rutu Rinott, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shiva Shankar, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Sung-Bae Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Kohler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Andrei Poenaru, Vlad T. Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xia Tang, Xiaofang Wang, Xiaojuan Wu, Xiaolan Wang, Xide Xia, Xilun Wu, Xinbo Gao, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu Wang, Yuchen Hao, Yundi Qian, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, and Zhiwei Zhao. 2024. The Llama 3 Herd of Models. *ArXiv abs/2407.21783* (2024).
- [8] Wenji Fang, Yao Lu, Shang Liu, Qijun Zhang, Ceyu Xu, Lisa Wu Wills, Hongce Zhang, and Zhiyao Xie. 2025. Transferable Presynthesis PPA Estimation for RTL Designs With Data Augmentation Techniques. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44 (2025), 200–213.
- [9] Farshad Firouzi, David Z Pan, Jiaqi Gu, Bahar Farahani, Jayeeta Chaudhuri, Ziang Yin, Pingchuan Ma, Peter Domanski, and Krishnendu Chakrabarty. 2025. ChipMnd: LLMs for Agile Chip Design. In *2025 IEEE 43rd VLSI Test Symposium (VTS)*. IEEE, 1–10.
- [10] Jiahao Gai, Hao Chen, Zhican Wang, Hongyu Zhou, Wanru Zhao, Nicholas Lane, and Hongxiang Fan. 2025. Exploring code language models for automated hls-based hardware generation: Benchmark, infrastructure and analysis. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. 988–994.

- [11] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. 2023. Large Language Models Are Zero-Shot Time Series Forecasters. *ArXiv abs/2310.07820* (2023).
- [12] Abdullah Guler and Niraj Kumar Jha. 2020. McPAT-Monolithic: An Area/Power/Timing Architecture Modeling Framework for 3-D Hybrid Monolithic Multicore Systems. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28 (2020), 2146–2156.
- [13] Hanxian Huang, Zhenghan Lin, Zixuan Wang, Xin Chen, Ke Ding, and Jishen Zhao. 2024. Towards LLM-Powered Verilog RTL Assistant: Self-Verification and Self-Correction. *ArXiv abs/2406.00115* (2024).
- [14] Donggyu Kim, Jerry Zhao, Jonathan Bachrach, and Krste Asanović. 2019. Simmani: Runtime Power Modeling for Arbitrary RTL with Automatic Signal Selection. *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (2019).
- [15] Ajay Krishna Ananda Kumar, Sami Al-Salamin, Hussam Amrouch, and Andreas Gerstlauer. 2023. Machine Learning-Based Microarchitecture-Level Power Modeling of CPUs. *IEEE Trans. Comput.* 72, 4 (April 2023), 941–956. <https://doi.org/10.1109/TC.2022.3185572>
- [16] Benjamin C Lee and David M Brooks. 2006. Accurate and efficient regression modeling for microarchitectural performance and power prediction. *ACM SIGOPS operating systems review* 40, 5 (2006), 185–194.
- [17] Benjamin C Lee and David M Brooks. 2007. Illustrative design space studies with microarchitectural regression models. In *2007 IEEE 13th International Symposium on High Performance Computer Architecture*. IEEE, 340–351.
- [18] Sheng Li, Jung Ho Ahn, Richard D. Strong, Jay B. Brockman, Dean M. Tullsen, and Norman P. Jouppi. 2009. McPAT: An Integrated Power, Area, and Timing Modeling Framework for Multicore and Manycore Architectures. In *2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 469–480.
- [19] Sheng Li, Jung Ho Ahn, Richard D. Strong, Jay B. Brockman, Dean M. Tullsen, and Norman P. Jouppi. 2013. The McPAT Framework for Multicore and Manycore Architectures: Simultaneously Modeling Power, Area, and Timing. *ACM Transactions on Architecture and Code Optimization* 10, 1 (April 2013), 1–29. <https://doi.org/10.1145/2445572.2445577>
- [20] Sheng Li, Ke Chen, Jung Ho Ahn, Jay B Brockman, and Norman P Jouppi. 2011. CACTI-P: Architecture-level modeling for SRAM-based structures with advanced leakage reduction techniques. In *2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 694–701.
- [21] Zhe Lin, Zike Yuan, Jieru Zhao, Wei Zhang, Hui Wang, and Yonghong Tian. 2022. PowerGear: Early-Stage Power Estimation in FPGA HLS via Heterogeneous Edge-Centric GNNs. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Antwerp, Belgium, 1341–1346. <https://doi.org/10.23919/DATES4114.2022.9774682>
- [22] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–8.
- [23] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. Rtlcoder: Fully open-source and efficient llm-assisted rtl code generation technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).
- [24] Tong Liu, Haoyu Zhao, and Yangdi Lyu. 2024. CAPEDL: Cycle-Accurate Power Estimation with Deep Learning. In *2024 2nd International Symposium of Electronics Design Automation (ISED)*. IEEE, Xi'an, China, 642–647. <https://doi.org/10.1109/ISED62518.2024.10617476>
- [25] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. Rtlm: An open-source benchmark for design rtl generation with large language model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 722–727.
- [26] Ruiyang Ma, Yuxin Yang, Ziqian Liu, Jiaxi Zhang, Min Li, Junhua Huang, and Guojie Luo. 2024. Verilogreader: Llm-aided hardware test generation. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–5.
- [27] Nowfel Mashnoor, Mohammad Akyash, Hadi Kamali, and Kimia Azar. 2025. TimelyHLS: LLM-Based Timing-Aware and Architecture-Specific FPGA HLS Optimization. *arXiv preprint arXiv:2507.17962* (2025).
- [28] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [29] Michael Popoloski. 2025. slang - SystemVerilog Language Services. <https://github.com/MikePopoloski/slang> Python bindings for slang, a library for compiling SystemVerilog. Accessed: 2025-05-02.
- [30] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [31] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners.
- [32] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2019. ZeRO: Memory optimizations Toward Training Trillion Parameter Models. *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis* (2019), 1–16.
- [33] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2020).
- [34] Fabian Stuckmann, Moritz Weißbrich, and Guillermo Payá-Vayá. 2024. Energy-Aware Register Allocation for VLIW Processors. *Journal of Signal Processing Systems* 96, 11 (Nov. 2024), 627–650. <https://doi.org/10.1007/s11265-024-01930-x>
- [35] Aoxiang Tang, Yang Yang, Chun-Yi Lee, and Niraj Kumar Jha. 2015. McPAT-PVT: Delay and Power Modeling Framework for FinFET Processor Architectures Under PVT Variations. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 23 (2015), 1616–1627.
- [36] Mingxin Tang, Wei Chen, Lizhou Wu, Libo Huang, and Kun Zeng. 2025. ChatDSE: A Zero-Shot Microarchitecture Design Space Explorer Powered by GPT4. 0. *ACM Transactions on Design Automation of Electronic Systems* (2025).
- [37] Hadi Tavakkoli, Shing Kai Lai, Wenhao Chen, Sile Fang, and Yi-Kuen Lee. 2025. Sensorless Power Measurement for Personal Computers Using CMOS MEMS Magnetometers and Machine Learning. In *2025 1st International Conference on Consumer Technology (ICCT-Pacific)*. IEEE, Matsue, Shimane, Japan, 1–4. <https://doi.org/10.1109/ICCT-Pacific63901.2025.11012908>
- [38] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [40] Hanyu Wang, Xinrui Wu, Zijian Ding, Su Zheng, Chengyue Wang, Tony Nowatzki, Yizhou Sun, and Jason Cong. 2025. LLM-DSE: Searching Accelerator Parameters with LLM Agents. *arXiv preprint arXiv:2505.12188* (2025).
- [41] Xi Wang, Gwok-Waa Wan, Sam-Zaak Wong, Layton Zhang, Tianyang Liu, Qi Tian, and Jianmin Ye. 2024. Chatcpu: An agile cpu design and verification platform with llm. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [42] Zhiyao Xie, Shiyu Li, Mingyuan Ma, Chen-Chia Chang, Jingyu Pan, Yiran Chen, and Jiangkun Hu. 2022. DEEP: Developing Extremely Efficient Runtime On-Chip Power Meters. *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)* (2022), 1–9.
- [43] Zhiyao Xie, Xiaoming Xu, Matt Walker, Joshua Knebel, K.J. Palaniswamy, Nicolas Hebert, Jiang Hu, Huanrui Yang, Yiran Chen, and Shidhartha Das. 2021. APOLLO: An Automated Power Modeling Framework for Runtime Power Introspection in High-Volume Commercial Microprocessors. *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (2021).
- [44] Chenwei Xiong, Cheng Liu, Huawei Li, and Xiaowei Li. 2024. HSLPilot: LLM-based High-Level Synthesis. *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design* (2024).
- [45] Lei Xu, Shanshan Wang, Emmanuel Casseau, and Chenglong Xiao. 2025. Intelligent4DSE: Optimizing high-level synthesis design space exploration with graph neural networks and large language models. *arXiv preprint arXiv:2504.19649* (2025).
- [46] Yinan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, et al. 2022. Towards developing high performance RISC-V processors using agile methodology. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1178–1199.
- [47] Jianwang Zhai, Chen Bai, Binwu Zhu, Yici Cai, Qiang Zhou, and Bei Yu. 2023. McPAT-Calib: A RISC-V BOOM Microarchitecture Power Modeling Framework. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 1 (Jan. 2023), 243–256. <https://doi.org/10.1109/TCAD.2022.3169464>
- [48] Jianwang Zhai, Yici Cai, and Bei Yu. 2023. Microarchitecture Power Modeling via Artificial Neural Network and Transfer Learning. *2023 28th Asia and South Pacific Design Automation Conference (ASP-DAC)* (2023), 1–6.
- [49] Qijun Zhang, Mengming Li, Yao Lu, and Zhiyao Xie. 2025. FirePower: Towards a Foundation with Generalizable Knowledge for Architecture-Level Power Modeling. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. ACM, Tokyo Japan, 1145–1152. <https://doi.org/10.1145/3658617.3697554>
- [50] Qijun Zhang, Shiyu Li, Guanglei Zhou, Jingyu Pan, Chen-Chia Chang, Yiran Chen, and Zhiyao Xie. 2023. PANDA: Architecture-Level Power Evaluation by Unifying Analytical and Machine Learning Solutions. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, San Francisco, CA, USA, 01–09. <https://doi.org/10.1109/ICCAD57390.2023.10323665>
- [51] Yanqing Zhang, Haoxing Ren, and Bruce Khailany. 2020. GRANNITE: Graph Neural Network Inference for Transferable Power Estimation. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, San Francisco, CA, USA, 1–6. <https://doi.org/10.1109/DAC18072.2020.9218643>
- [52] Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Muxin Song, Yinan Xu, Ziyuan Nan, Mingju Gao, Tianyun Ma, Lei Qi, et al. 2024. CodeV: Empowering LLMs with HDL Generation through Multi-Level Summarization. *arXiv preprint arXiv:2407.10424* (2024).

- [53] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics, Bangkok, Thailand. <http://arxiv.org/abs/2403.13372>
- [54] Yuan Zhou, Haoxing Ren, Yanqing Zhang, Ben Keller, Bruce Khailany, and Zhiru Zhang. 2019. PRIMAL: Power Inference Using Machine Learning. In *Proceedings of the 56th Annual Design Automation Conference 2019*. ACM, Las Vegas NV USA, 1–6. <https://doi.org/10.1145/3316781.3317884>